

INSTITUTO SUPERIOR TECNOLÓGICO

TECSUP



DEPARTAMENTO DE TECNOLOGÍA DIGITAL

Diseño y desarrollo de software

“Diseño e implementación de una arquitectura distribuida con Docker
Swarm para la plataforma TechRetail”

ESTUDIANTES

Medina Mallqui Ailyn Madeley

Ochoa Marín Yamile Thais

DOCENTE

Jaime M. Farfán Madariaga

Lima Perú

2026 - 1

INDICE

Contenido

INTRODUCCIÓN	2
1. Arquitectura Implementada.....	3
1.1. Infraestructura del cluster	3
1.2. Servicios del Stack TechRetail	3
1.3. Seguridad y Configuración	4
1.4. Diagrama de la Arquitectura	5
2. Comandos Utilizados.....	6
2.1. Inicialización del Clúster	6
2.2. Creación del Secret y el Config	7
2.3. Crear carpeta.....	7
2.4. Despliegue del Stack.....	9
2.5. Escalado Dinámico.....	9
2.6. Visualización del Clúster	10
3. Problemas y soluciones.....	11
4. Reflexión sobre Docker Swarm	12
4.1. Ventajas Comprobadas en la Práctica	12
4.2. Aprendizajes del Equipo.....	13
Video demostrativo	13
	1

INTRODUCCIÓN

TechRetail es una compañía de e-commerce que, a raíz de su expansión, empezó a lidiar con problemas críticos en su plataforma, como fallos constantes del sistema, largos tiempos de respuesta y restricciones para expandirse frente a picos de demanda elevada.

Para resolver estos problemas, se sugiere la puesta en marcha de una arquitectura fundamentada en microservicios, empleando Docker Swarm como herramienta de orquestación, lo que facilita la administración eficaz de contenedores en un ambiente distribuido.

Este reporte detalla la arquitectura puesta en marcha, el procedimiento de configuración del clúster, los comandos empleados para la implementación de los servicios, además de los problemas más relevantes hallados durante la implementación y sus correspondientes soluciones. Finalmente, se expone una consideración acerca de la utilización de Docker Swarm como instrumento para asegurar la escalabilidad y la disponibilidad elevada en aplicaciones contemporáneas.

1.Arquitectura Implementada

1.1. Infraestructura del cluster

Debido a la discontinuación de la plataforma Play with Docker el 22 de marzo de 2026 y que no nos garantiza estabilidad para el proyecto, se optó por crear el entorno usando VMware Workstation con tres máquinas virtuales, todas corriendo Ubuntu Server 22.04 LTS con Docker Engine instalado manualmente.

Nodo	Rol	Sistema Operativo
swarm-manager	Manager / Leader	Ubuntu Server 22.04 LTS
swarm-worker-1	Worker	Ubuntu Server 22.04 LTS
swarm-worker-2	Worker	Ubuntu Server 22.04 LTS

El nodo manager se desempeña como el núcleo del clúster, recibe las órdenes del operador, sostiene el estado ideal del sistema y reparte las labores entre los workes . Los workes son responsables de llevar a cabo los contenedores que se les asignan.

1.2. Servicios del Stack TechRetail

Se establecieron cinco servicios en el archivo docker-compose. yml, todos vinculados mediante una red overlay segura (techretail_net) que facilita la comunicación entre los contenedores sin importar el nodo físico donde se encuentren.

Servicio	Imagen	Réplicas	Descripción
frontend	nginx:alpine	3	Servidor web. 3 réplicas aseguran accesibilidad en caso de que un nodo falle.
backend	node:18-alpine	2	API de la aplicación. Usa el Secret y el Config del clúster.
database	mysql:8	1 (manager)	Base de datos. Restringida al manager para acceso al volumen.
cache	redis:7-alpine	1	Caché en memoria para reducir carga en la base de datos.
visualizer	dockersamples/ visualizer	1 (manager)	Panel web que muestra la distribución del clúster en tiempo real.

La base de datos y el visualizador están configurados para ejecutarse exclusivamente en el nodo manager (el nodo principal del clúster). La razón es porque la base de datos usa un almacenamiento persistente (db_data) que está guardado en ese mismo nodo. Así garantizamos que la base de datos siempre pueda acceder a su información de manera constante, ya que si se ejecutara en otro nodo no tendría acceso a ese almacenamiento.

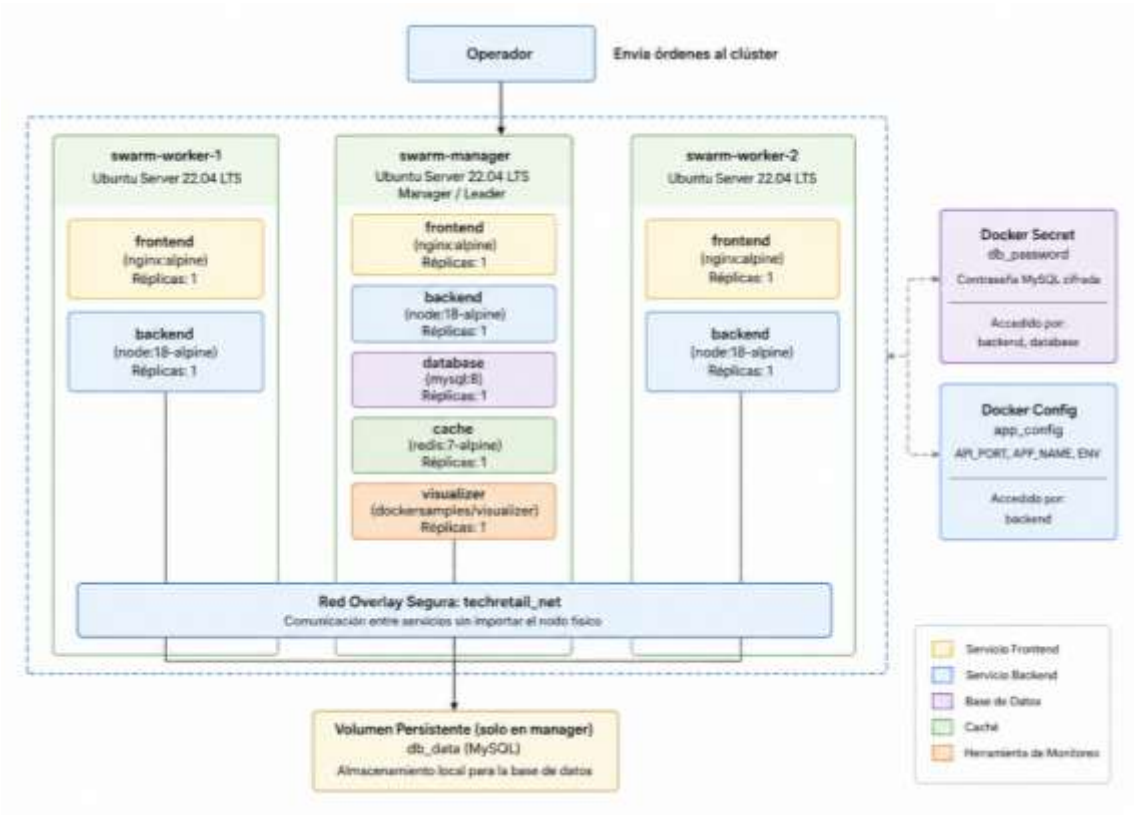
1.3. Seguridad y Configuración

Con el fin de manejar información sensible, se empleó Docker Secret, que se encarga de encriptar la contraseña de la base de datos tanto en tránsito como en reposo, integrándola como un archivo en el contenedor (/run/secrets/db_password). Para configuraciones que no requieren

protección, se utilizó Docker config, el cual guarda las variables de entorno de la aplicación (puerto, nombre y entorno de ejecución) directamente en el clúster, accesibles para los contenedores en funcionamiento.

Recurso	Nombre	Contenido	Accedido por
Docker Secret	db_password	Contraseña MySQL cifrada	backend, database
Docker Config	app_config	API_PORT, APP_NAME, ENV	backend

1.4. Diagrama de la Arquitectura



2. Comandos Utilizados

2.1. Inicialización del Clúster

Primero inicializamos el swarm en el nodo manager. El flag `--advertise-addr` indica la IP que los workes usarán para conectarse:

```
# Ejecutamos en swarm-manager
docker swarm init --advertise-addr 192.168.163.135
```

```
manager@swarm-manager:~$ docker swarm init --advertise-addr 192.168.163.135
Swarm initialized: current node (scilhdwze53lqlg6xfetvs2ey) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2x8p5t0frctbykde9zkkfeuenz02xb1j5e3isjbsk232ratzdgg-Bn55iygikx9o6tp63urthbr13j 192.168.163.135:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
manager@swarm-manager:~$ _
```

Este comando genera automáticamente un token de incorporación. La salida incluye el comando exacto para unir workers. En cada nodo worker se ejecutó:

```
# Ejecutar en swarm-worker-1 y swarm-worker-2
docker swarm join --token SWMTKN-1-ABCDEF... 192.168.1.25:2377
```

Para confirmar que los tres nodos están operativos y en buen estado:

```
docker node ls
```

Como resultado tenemos 3 nodos en estado ready, teniendo a swarm-manager como Leader.

```
manager@swarm-manager:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	EN
scilhdwze53lqlg6xfetvs2ey *	swarm-manager	Ready	Active	Leader	29
zkh0shhe59qyitzkc5kfvxg60	swarm-worker	Ready	Active		29
exnfahi7ql9l0wcp6p40jd2t1	swarm-worker-1	Ready	Active		29

```
manager@swarm-manager:~$
```

2.2. Creación del Secret y el Config

Estos recursos deben crearse antes del despliegue, ya que el compose los declara como externos:

```
# Crear el Secret (contraseña cifrada de la BD)
echo "MiPasswordSegura123" | docker secret create db_password -
```

```
manager@swarm-manager:~$ echo "manager123" | docker secret create db_password -
yzuz8jalbiss7i2pik0cnn4qr
manager@swarm-manager:~$ docker secret ls


| ID                        | NAME        | DRIVER | CREATED       | UPDATED       |
|---------------------------|-------------|--------|---------------|---------------|
| yzuz8jalbiss7i2pik0cnn4qr | db_password |        | 7 seconds ago | 7 seconds ago |


manager@swarm-manager:~$
```

```
# Crear el Config (parámetros del backend)
docker config create app_config configs/app_config.env
```

```
manager@swarm-manager:~$ mkdir configs
manager@swarm-manager:~$ nano configs/app_config.env
manager@swarm-manager:~$ docker config create app_config configs/app_config.env
r0f78qbh8wx34irkwgssiukev
manager@swarm-manager:~$ docker config ls


| ID                        | NAME       | CREATED       | UPDATED       |
|---------------------------|------------|---------------|---------------|
| r0f78qbh8wx34irkwgssiukev | app_config | 5 seconds ago | 5 seconds ago |


manager@swarm-manager:~$
```

El guión al final del primer comando para crear Secret indica que el valor se lee desde la entrada, evitando así que la contraseña quede guardada en el historial de comandos de la terminal.

2.3. Crear carpeta

Se preparó el entorno del proyecto creando la carpeta principal y el archivo de configuración donde se define la pila de servicios.

```
mkdir techretail
cd techretail
nano docker-compose.yml
```


Dentro del editor se pegó el contenido del archivo docker-compose.yml, disponible en el repositorio oficial del proyecto en GitHub: <https://github.com/lynmad05/techretail-docker-swarm.git>

Para guardar y salir del editor:

CTRL + O → ENTER → CTRL + X

```

manager@swarm-manager: ~/TechRetail
GNU nano 7.2
version: "3.8"

services:
  frontend:
    image: nginx:alpine
    ports:
      - "80:80"
    networks:
      - techretail_net
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure

  backend:
    image: node:18-alpine
    command: node -e "require('http').createServer((req,res)=>res.end('TechRetail API OK')).listen(3000)"
    networks:
      - techretail_net
    deploy:
      replicas: 2
      restart_policy:
        condition: on-failure
    secrets:
      - db_password
    configs:
      - app_config

  database:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password
      MYSQL_DATABASE: techretail_db
    networks:
      - techretail_net
    volumes:
      - db_data:/var/lib/mysql
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager
    secrets:
      - db_password

  cache:
    image: redis:7-alpine
    networks:
      - techretail_net
    deploy:
      replicas: 1

  visualizer:
    image: dockersamples/visualizer

```

2.4. Despliegue del Stack

Con el clúster listo y los recursos previamente creados, se desplegó la pila completa de servicios:

```
# Desplegar el stack desde el archivo compose
docker stack deploy -c docker-compose.yml techretail
```

```
# Verificar que todos los servicios están corriendo
docker stack services techretail
```

```
manager@swarm-manager:~/techretail$ docker stack services techretail
ID                NAME                MODE                REPLICAS    IMAGE                PORTS
snkvyfbmxgj1     techretail_backend  replicated          1/2         node:l8-alpine
jkgrnuj2a6p9     techretail_cache    replicated          1/1         redis:7-alpine
x6mc7uv6andn     techretail_database replicated          0/1         mysql:8
uxfvbbwj9y6h     techretail_frontend replicated          2/3         nginx:alpine        *:80->80
/tcp
lv7iqmnculec     techretail_visualizer replicated          0/1         dockersamples/visualizer:latest *:8080->8080/tcp
manager@swarm-manager:~/techretail$
```

```
# Ver en qué nodo está cada contenedor de un servicio
docker service ps techretail_frontend
```

```
manager@swarm-manager:~/techretail$ docker service ps techretail_frontend
ID                NAME                IMAGE                NODE                DESIRED STATE    CURRENT STATE    ERROR
PORTS
yr9ur9qs3eko     techretail_frontend.1 nginx:alpine        swarm-manager      Running          Running 40 seconds ago
g3864871ho1u     techretail_frontend.2 nginx:alpine        swarm-worker-1     Running          Running 45 seconds ago
732k7yqg30qd     techretail_frontend.3 nginx:alpine        swarm-worker       Running          Running 45 seconds ago
manager@swarm-manager:~/techretail$
```

2.5. Escalado Dinámico

Una de las características más significativas de Swarm es la capacidad de modificar el conteo de réplicas en tiempo real, sin detener el servicio. Esto se evidenció al aumentar el número de instancias del frontend de 3 a 5 réplicas.

```
docker service scale techretail_frontend=5
```

Swarm distribuyó automáticamente las 2 nuevas réplicas entre los nodos disponibles. Para validar la distribución después del escalado:

```
docker service ps techretail_frontend
```

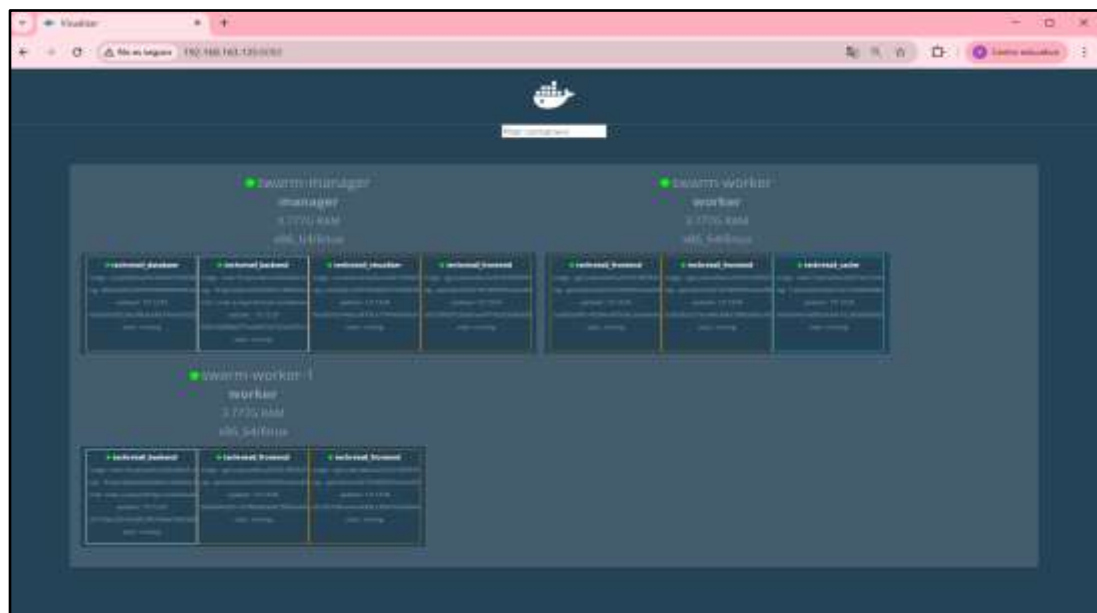
```
manager@swarm-manager:~/techretail$ docker service scale techretail_frontend=5
techretail_frontend scaled to 5
overall progress: 5 out of 5 tasks
1/5: running [=====]
2/5: running [=====]
3/5: running [=====]
4/5: running [=====]
5/5: running [=====]
verify: Service techretail_frontend converged
manager@swarm-manager:~/techretail$ docker service ps techretail_frontend
ID                NAME                IMAGE                NODE                DESIRED STATE       CURRENT STATE            ERROR             PORTS
7S                techretail_frontend.1  nginx:alpine        swarm-manager       Running              Running about a minute ago
5564871h0ls       techretail_frontend.2  nginx:alpine        swarm-worker-1      Running              Running about a minute ago
732t7yp339gd      techretail_frontend.3  nginx:alpine        swarm-worker         Running              Running about a minute ago
86ekp0wv845k      techretail_frontend.4  nginx:alpine        swarm-worker-1      Running              Running 8 seconds ago
8hpoe1p89tm       techretail_frontend.5  nginx:alpine        swarm-worker         Running              Running 8 seconds ago
manager@swarm-manager:~/techretail$
```

2.6. Visualización del Clúster

El servicio visualizer se activó de forma automática junto con el stack. Se logró el acceso a través del navegador con:

```
http://192.168.163.13:8080
```

La repartición de cada uno de los contenedores entre los tres nodos del clúster se visualizó en tiempo real desde allí.



3.Problemas y soluciones

Tres problemas específicos se encontraron durante la implementación. Cada uno se documenta con su respectiva causa y la solución que se implementó:

#	Problema	Causa	Solución Aplicada
1	Play with Docker no disponible	La plataforma no cargaba correctamente y no ofrecía la estabilidad necesaria para el trabajo	Se migró completamente a VMware Workstation, creando 3 VMs con Ubuntu Server 22.04 LTS e instalando Docker manualmente
2	Error: secret 'db_password' not found al hacer deploy	El compose referencia el secret como external: true, pero aún no había sido creado en el clúster	Se creó el secret previamente con: <code>echo '...' docker secret create db_password -</code>
3	Error: config 'app_config' not found al hacer deploy	Igual que el caso anterior: el config se declara externo pero no existía en el swarm	Se creó el config antes del deploy con: <code>docker config create app_config configs/app_config.env</code>

La falta de disponibilidad de Play with Docker fue el mayor impedimento. A pesar de que usar VMware involucró más pasos para la configuración inicial, se obtuvo un ambiente más estable y duradero (las VMs no desaparecen después de 4 horas

como con Play with Docker), lo cual ayudó a realizar las pruebas iterativas del despliegue.

4. Reflexión sobre Docker Swarm

4.1. Ventajas Comprobadas en la Práctica

- Alta disponibilidad real: Swarm redistribuye las tareas automáticamente entre los nodos si un nodo falla, gracias a que hay tres réplicas del frontend repartidas entre ellos. TechRetail dejaría de experimentar interrupciones totales del sistema.
- Sin tiempo de inactividad, escalado en segundos: Con un solo comando, se pasó de 3 a 5 réplicas del frontend en unos pocos segundos. Ninguna conexión activa se cortó. TechRetail solucionaría sus picos de demanda en este lugar preciso.
- Manejo seguro de credenciales: Docker Secrets suprimió la necesidad de escribir contraseñas en variables de entorno visibles o en el código. En ningún archivo del repositorio se muestra la contraseña de MySQL en texto plano.
- Infraestructura como código: El archivo docker-compose.yml documenta y reproduce toda la arquitectura. Cualquier persona del equipo puede levantar el entorno completo con dos comandos.
- Curva de aprendizaje accesible: Docker Swarm reutiliza el formato Compose que los desarrolladores ya conocen. La transición desde Docker Compose local a un clúster distribuido fue directa.

4.2. Aprendizajes del Equipo

Este trabajo proporcionó lecciones específicas que trascienden la teoría:

- Comprender la distinción verdadera entre un servicio orquestado y un contenedor aislado. Un servicio en Swarm no es simplemente un contenedor, sino una unidad con réplicas, políticas de reinicio, limitaciones de localización y comunicación confiable.
- La relevancia del orden de las operaciones en la infraestructura. Entender que los Secrets y Configs tienen que estar presentes antes del despliegue fue más útil a través de la experiencia que por medio de la lectura.
- Que la adaptación técnica ante obstáculos (migrar de Play with Docker a VMware) es una habilidad profesional tan valiosa como el dominio de los comandos mismos.
- Docker Swarm resuelve de forma directa los cuatro problemas de TechRetail: las réplicas eliminan las caídas totales, la red overlay resuelve la comunicación entre servicios, el escalado atiende los picos de demanda, y los Secrets protegen los datos sensibles.

Video demostrativo

El despliegue completo del sistema puede visualizarse en el siguiente enlace:

<https://drive.google.com/file/d/1E2NvZhnxQb8Im9CUaiYyjWH61BjAFxBB/view?usp=sharing>